Caesar Cipher — Assembly Language Project

Course: Programming Paradigms
Institution: University of Pécs
Programming Language: x86-64 Assembly (NASM)
Author: Hugo Lugo
Project: Final Assignment

1. Introduction

The objective of this project is to implement the Caesar Cipher encryption algorithm using x86-64 Assembly language. The program provides a command-line interface through which the user can encrypt or decrypt a message using a customizable shift value.

The project demonstrates fundamental Assembly programming concepts, including register manipulation, memory addressing, loops, conditional jumps, procedures, and interaction with the operating system through Linux system calls.

2. Caesar Cipher Algorithm

The Caesar Cipher is a classical substitution cipher in which each letter is replaced by another letter located a fixed number of positions away in the alphabet.

For example, using a shift of 3:

- A becomes D.
- B becomes E.
- X becomes A.
- Z becomes C.

Encryption moves letters forward, while decryption moves them backward.

The mathematical expressions are:

Encryption: E(x) = (x + k) mod 26

Decryption: D(x) = (x - k + 26) mod 26

Where *x* is the letter's position in the alphabet (0–25), and *k* is the shift value.

The implementation handles alphabet wrap-around using comparisons and adjustments of 26 positions rather than performing a modulo operation.

3. Program Implementation

The program is divided into three main sections:

.data: Stores initialized text messages, including prompts, the operation menu, and error messages.

.bss: Reserves memory for the user's operation choice, shift input, and message buffer.

.text: Contains the executable instructions, encryption and decryption logic, and the output procedure.

3.1 User Input and Validation

The program first asks the user to select encryption or decryption by entering 1 or 2.

Next, the user enters a shift value between 0 and 25. Since the value is received as ASCII text, the program converts each digit into its numerical equivalent.

Basic validation rejects invalid operation selections and shift values outside the permitted range.

Finally, the program reads the message into a 256-byte buffer.

3.2 Encryption and Decryption

The program processes the message character by character using a loop.

For each character:

1. The character is loaded from the input buffer.
2. Its ASCII value is compared against the uppercase and lowercase letter ranges.
3. If it is a letter, the selected shift is applied in the appropriate direction.
4. If the result falls outside its alphabetic range, wrap-around is performed by adding or subtracting 26.
5. The resulting character is stored back into the buffer.

Characters outside the ASCII alphabetic ranges are left unchanged.

This technique performs the transformation directly in the original buffer, avoiding the need for additional memory to store the encrypted message.

3.3 Registers and System Calls

The implementation uses several registers for specific purposes:

Register	Purpose
RAX	System call selection and return values
RDI	Input/output file descriptor or exit status
RSI	Memory address of input/output data
RDX	Number of bytes to read or write
RCX	Character-processing loop counter
R12	Number of message bytes read
R13	Selected shift value
R14B	Encryption or decryption selection

Linux system calls are used for reading input, writing output, and terminating the program.

A reusable `print_text` procedure reduces repeated instructions when displaying messages.

4. Testing

The program was tested with different messages and shift values to verify its functionality.

Operation	Shift	Input	Expected Output
Encrypt	3	Hello World!	Khoor Zruog!
Decrypt	3	Khoor Zruog!	Hello World!
Encrypt	3	XYZ	ABC
Decrypt	3	ABC	XYZ
Encrypt	13	Abc XYZ	Nop KLM
Encrypt	0	Hello 123!	Hello 123!

These cases cover basic encryption and decryption, alphabet wrap-around, custom shifts, and preservation of non-alphabetic characters.

The implementation uses fixed-size buffers and is intended for ordinary terminal input. Handling of oversized input is a known limitation.

5. Conclusion

The project demonstrates how a classical encryption algorithm can be implemented using Assembly language without relying on high-level string-processing functions.

Developing the program provided practical experience with ASCII character manipulation, register usage, memory operations, conditional jumps, and system calls.

Although the Caesar Cipher does not provide modern cryptographic security, it serves as a useful educational example for understanding low-level programming and algorithm implementation.